CHAPTER 1

DESIGNING A HOTEL MANAGEMENT SYSTEM

1 Getting Ready: The Hotel Management System 2
2 Requirements for the Hotel Management System 4
3 Use Case Diagram for the Hotel Management System . . . 5
4 Class Diagram for the Hotel Management System 8
5 Sequence Diagram for the Hotel Management System . 20
6 Activity Diagram for the Hotel Management System . . . 23
7 Code for the Hotel Management System 26
8 Summary: Designing a Hotel Management System 36
9 Key Insights: The Hotel Management System 40

This chapter covers the design of a hotel management system, detailing the requirements, UML diagrams, and implementation code for a complete system.

1 Getting Ready: The Hotel Management System

1.1 Problem definition

The **hotel management system** is a software used to manage all hotel activities efficiently and smoothly. Almost all popular hotels have an online management system to digitize room bookings, manage staff, and perform other necessary hotel management features. This system makes the notification and payment process flexible and automated. The hotel usually has a fixed number of rooms, and the booking system has information about all rooms present and their availability.
For the hotel manager, manually controlling all the hotel activities is difficult. With the help of an online hotel management system, the manager can keep track of rooms, customers, and workers through a single portal. The manager can also post available rooms to the system and generate bills. The system also allows customers to search for and book a room of their choice and cost range. The system provides complete information about rooms to the customers so they can view the available rooms and book them online. The guest is charged based on the time the hotel room is booked. In this way, the system facilitates the hotel's customers and managers.

1.2 Expectations from the interviewee

Numerous components are present in the hotel management system, each with specific constraints and requirements. The following provides an overview of some of the main things the interviewer will want to hear you discuss in more detail.

Room booking

Booking a room is an essential part of the hotel management system. The system must ensure that no two users can book the same room for overlapping time slots. The interviewer expects you to ask questions to identify how the system will work in such situations:

- How will the system ensure multiple users do not book the same room?
- What type of users are allowed to book a room in the hotel?
- Can users book a room in advance?

Payment handling

One of the hotel management system's most significant attributes is its payment structure for its customers. This can vary, so the interviewer would expect you to ask the questions listed below:

- What payment methods can the customer use (for example, credit card or cash)?
- How is the payment performed? Does the customer pay online or through a receptionist at the hotel?
- Will the customer be able to pay in advance for a room booking, or is a just-in-time (JIT) payment system available?

Price variance

We touched upon the payment methods of the hotel management system. Now , the pricing model needs to be clarified by the interviewer. Therefore, you may ask the questions listed below:

- How will the booking price be calculated? Or which factors affect the price of a room in the hotel?
- How does the location and size of the room affect its price?
- How does the booking duration affect the payment?

Booking cancellation

There will be many duplicate instances in our system. The interviewer expects you to ask the questions provided below:

- Can the user cancel a room booking?
- Which type of users are allowed to request a room booking cancellation?

1.3 Design approach

We'll design this hotel management system using the bottom-up design approach. For this purpose, we will follow the steps below:

- Identify and design the smallest components first, such as a room.
- Use these small components to design bigger components, such as building a hotel with multiple rooms.
- Repeat the steps above until we design the whole hotel management system.

1.4 Design pattern

During an interview, discussing the design patterns under which the hotel management system falls is always a good practice. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design.
Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking, "Define design patterns."

Question:

Which design pattern(s) should be used to design a hotel management system? Please elaborate on your choice(s).

Let's explore the requirements of the hotel management system in the next lesson.

2 Requirements for the Hotel Management System

In this lesson, we'll list the requirements of the hotel management system. This is a crucial step since requirements define the scope of a problem. Getting them right from the interviewer and understanding them well will make the design of the rest of the system smooth and easy. We'll use the notational convention to identify each requirement with a unique label "Rn," where "R" is short for requirement and "n" is a natural number.

2.1 Requirement collection

The requirements for the hotel management system are defined below:

R1: The system can have four types of accounts, such as housekeeper, receptionist, and guest.
R2: The rooms can be of different styles, such as standard, deluxe, family suite, or business suite.

Column component type 'MxGraphWidget' not yet supported.

R3: The system should allow guests to search for and book any available room.

Column component type 'MxGraphWidget' not yet supported.

R4: During room booking, the user enters the check-in date and the duration of the stay. The user also has to make an advance payment.

R5: The customer can cancel the booking; a full refund will be provided if the booking is canceled before 24 hours of check-in time.

R6: The system should notify customers about the booking status or other information.

R7: The system should log and manage all the cleanup tasks.

R8: The system should allow customers to add services of their choice, such as room service, food or kitchen service, or amenities.

Column component type 'MxGraphWidget' not yet supported.

R9: Every room should have its specific key, and there can be a master key that opens a specific set of rooms.

R10: A hotel can have multiple branches.

We've identified our requirements for the problem, and in the next lesson, we will define different use cases for the hotel management system.

3 Use Case Diagram for the Hotel Management System

Let's build the use case diagram for the hotel management system and understand the relationship between its different components.
First, we'll define the different elements of our hotel, followed by the complete use case diagram of the system.

3.1 System

Our system is a hotel management system.

3.2 Actors

Now, we will define the main actors of our hotel management system.

Primary actors

- **Guest:** This is the hotel's primary actor who can book a room, make a payment, and change or cancel the reservations.
- **Receptionist:** This actor acts as the system's admin and can perform any task a "Guest" can perform. It can also add, remove, or update the room, check in/check out guests, and issue room keys for guests.
- **Housekeeper:** This can add or update the room's maintenance status.

Secondary actors

- **System:** This can send booking notifications to guests.
- **Server:** This can add or update the room status according to the room change request.

3.3 Use cases

In this section, we will define the hotel's use cases. We have listed the use cases according to their interactions with a particular actor.

> **Note:** Some use cases will occur multiple times because they are shared among different actors in the system.

Guest

- **Book room:** To book a room in the hotel.
- **Update booking:** To update a room booking in the hotel.
- **Login/Logout:** To log in and out of the hotel management system.
- **Cancel booking:** To cancel a room booking in the hotel.
- **View booking:** To view and verify a room booking.
- **Print booking:** To print booking details from the hotel management system.
- **Search room/booking:** To search for a room or a booking in the hotel management system.
- **Payment:** To pay the room rent to the hotel.
- **View account:** To view account details and booking status.
- **Register new account:** To register a new account for new guests.
- **Return room key:** To return the room key before checkout.

Receptionist

- **Add room:** To add rooms to the hotel management system so guests can book them.
- **Update room:** To update the room status from available to booked or vice versa.
- **Remove room:** To remove a room from the hotel management system so guests can't book it.
- **Book room:** To book a room in the hotel.
- **Update booking:** To update a room booking in the hotel.
- **Login/Logout:** To log in and out of the hotel management system.
- **Cancel booking:** To cancel a room booking in the hotel.
- **View booking:** To view and verify a room booking.
- **Print booking:** To print booking details from the hotel management system.
- **Search room/booking:** To search for a room or a booking in the hotel management system.
- **View account:** To view account details and booking status.
- **Register new account:** To register a new account for new guests.
- **Check in guest:** To check in guests at the hotel.
- **Check out guest:** To check out guests from the hotel.
- **Issue room key:** To issue room keys to guests who have checked in.

Hotel system

- **Add/update room charge:** To update the status of the room charge.
- **Send booking notification:** To send a booking notification to guests.

Housekeeper

- **Add/update room maintenance:** To update the maintenance status of rooms.

3.4 Relationships

This section describes the relationships between and among actors and their use cases.

Associations

The table below shows the association relationship between actors and their use cases.

Guest	Receptionist	System	Housekeeper
Book room	Book room	Send booking notification	Add/update room maintenance
Payment	View account	Add/update room charge	
View account	Register new account		
Register new account	Print booking		
Print booking	Cancel booking		
Cancel booking	Login/Logout		
Login/Logout	Check in guest		
Search room/booking	Issue room key		
Update booking	Search room/booking		
View booking	Update booking		
Return room key	Check out guest		
	View booking		
	Add room		
	Remove room		
	Update room		

Include

When a guest books a room, the payment will be processed.

- The "Book room" use case includes the "Payment" use case.

When a receptionist checks in a guest, a key is issued to the guest.

- The "Check in guest" use case includes the "Issue room key"use case.

When a guest checks out, the key is returned to the receptionist.

- The "Check out guest" use case includes the "Return room key"use case.

If a booking is canceled, the payment will be refunded.

- The "Cancel booking" use case includes the "Refund payment"use case.

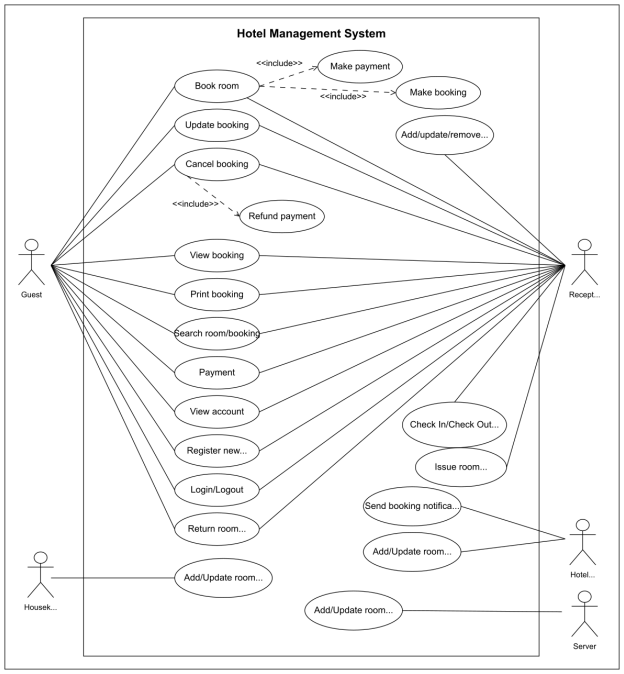
3.5 Use case diagram

Here's the use case diagram of the hotel management system:

In the next lesson, we'll discuss the class diagram with a detailed explanation of all classes and their relationship with each other.

4 Class Diagram for the Hotel Management System

Here, we will create the class diagram for our system based on the requirements we gathered in one of the previous lessons. In the class diagram, we will first design and create the classes, abstract classes, and interfaces for the system, and then we'll identify the relationship between classes in accordance with all the requirements of the hotel management system.



4.1 Components of a hotel management system

In this section, we'll define the classes for a hotel management system. Following the bottom-up approach to designing a class diagram, we will first create the classes of small components. Next , we will integrate these components and create the class diagram for the hotel management system.

Address and Account

The `Address` class is required to store any address. It is a custom data type with attributes like a street address, city, etc. In the hotel management system, this class will be used to specify the addresses of the users and the hotel.
`Account` is a class used to store the user's account information. This class has three members, i.e., account ID, password, and account status. The class representation of `Address` and `Account` classes is as follows:

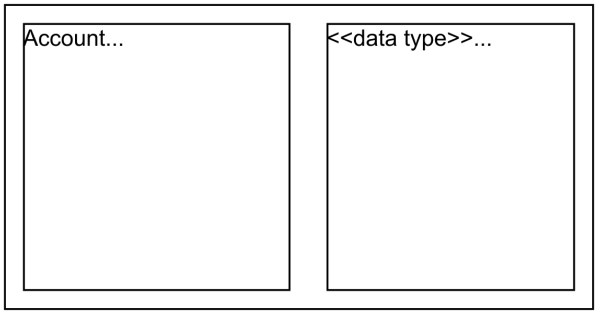


Figure 1.1: The class diagram of the Address and Account classes

Person

`Person` is an abstract class used to store information related to a person, such as a name, email, phone number, etc. In this class, an object of the `Address` type specifies the person's address. The `Person` class specifies the accounts in the system. The system has four types of accounts: housekeeper, receptionist, guest, and server.
There are multiple functions of the `Person` class's subclasses. First , the `Housekeeper` class will keep track of the maintenance records of a room. Second , the `Receptionist` class represents the hotel receptionist. The methods in this class depict the actions that the receptionist can perform. Moreover , the `Guest` class describes the guests of the hotel. Guests are the hotel's customers who can search for and book a room.
The relationship diagram for these classes is shown below:

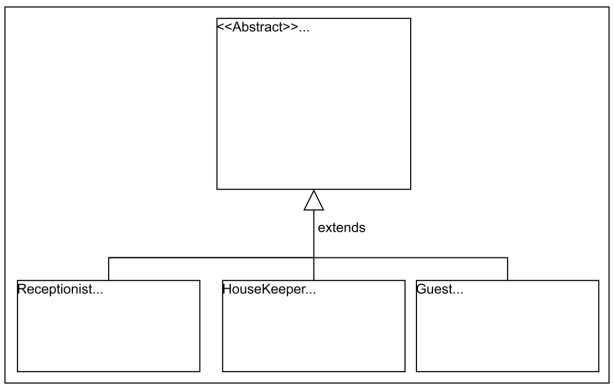


Figure 1.2: The class diagram of Person and its derived classes

R1: The system can have four types of accounts, such as housekeeper, receptionist, guest, or server.

Service

`Service` is an abstract class that encapsulates the details of different types of services guests have requested. Three types of services are provided: amenity, room service, and kitchen service.
The `Amenity` class is a subclass of `Service` with two members: name and description. Similarly , `RoomService` is also inherited from the `Service` class. This class stores information about room services, whether these services are chargeable or not, and the request time of the service. Furthermore , the last child class of the `Service` is `KitchenService`. The relationship between these classes is shown in the illustration below.

R8: The system should allow customers to add services of their choice, such as room service, food or kitchen service, or amenities.

Invoice

The `Invoice` class represents the billing system in the hotel management system. The UML diagram for both classes is presented below:

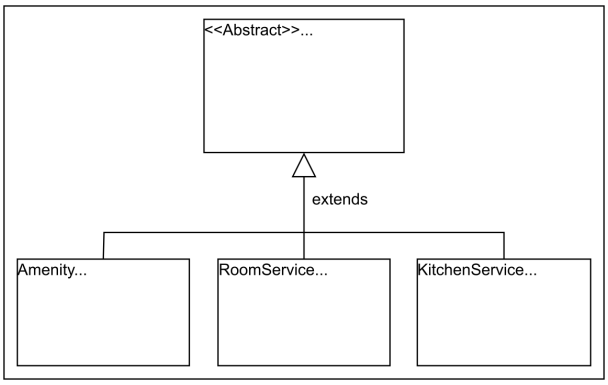


Figure 1.3: The class diagram of Service and its derived classes

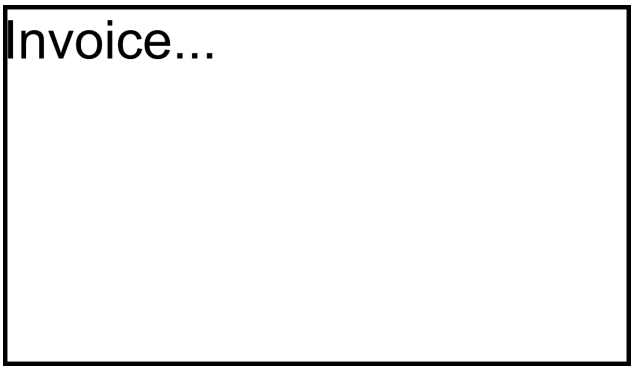


Figure 1.4: The class diagram of the Invoice class

Room booking

The `RoomBooking` class is responsible for managing the bookings for a room. This class includes attributes like reservation number, start date, duration, etc. Moreover , this class has a member of the `BookingStatus` type that is used to store the status of the room booking. The UML representation of this class is as follows:

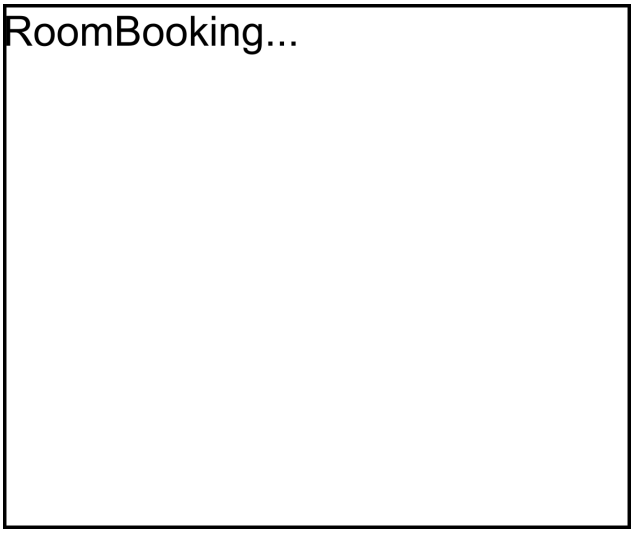


Figure 1.5: The class diagram of the RoomBooking class

R4: During room booking, the user enters the check-in date and duration of the stay. The user also has to make an advance payment.

Notification

`Notification` is an abstract class responsible for sending notifications to guests whenever the booking is nearing the check-in or check-out date. Every notification has an ID, creation date, and content. The notification can either be an SMS notification or an email notification.
The `SMSNotification` class requires the member's phone number to send a notification. On the other hand, the `EmailNotification` needs the member's email address to send a notification. The relationship diagram of these classes is shown here:

R6: The system should notify customers about the booking status or other information.

Room, room key, and room maintenance

The `Room` class is the system's basic building block. Every room has a room number and a price associated with it. The `Room` class uses the `RoomStyle` and `RoomStatus` enums to specify the rooms' style and status, respectively.
Each room has an electronic key card associated with it. The `RoomKey` class expresses the electronic key card. Each card has its own unique ID and barcode. The `RoomKey` class also has members to store the issue date, to check whether or not the key is active, and to check whether or not a key is a master key. Whereas , `RoomMaintenance` is a class used to keep

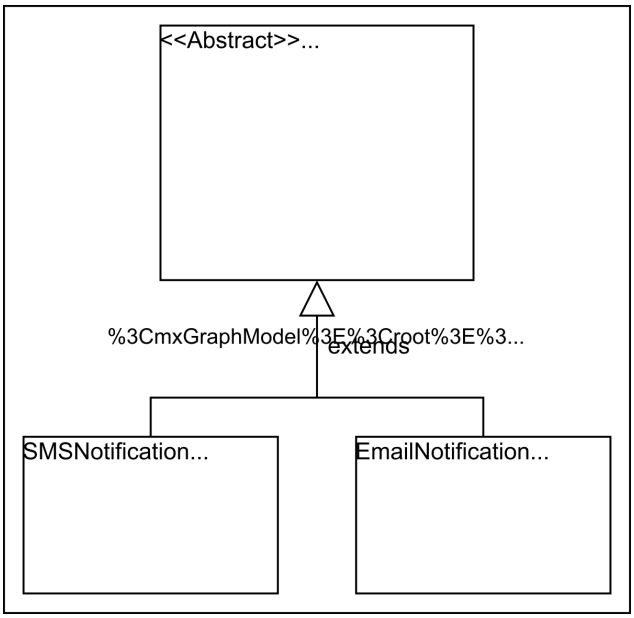


Figure 1.6: The class diagram of Notification and its derived classes

track of all maintenance records for the rooms. The UML representation of these classes is as follows:

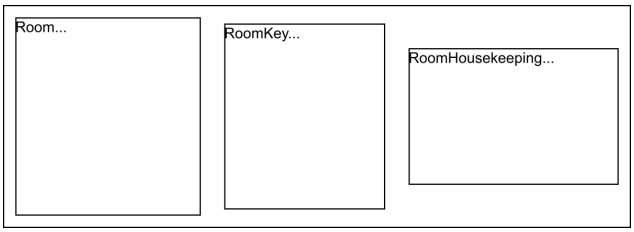


Figure 1.7: The class diagram of the Room, RoomKey, and RoomMaintenance classes

R9: Every room should have its specific key, and there can be a master key that opens a specific set of rooms.
R7: The system should log and manage all the cleanup tasks.

Search interface and catalog

`Search` is one of the most important components of the hotel management system. In the diagram below, `Search` is the interface that allows guests to search for any room of their choice and pay range. The receptionist can also use this interface to search for any room. The `Catalog` class contains a list of all rooms and implements the `Search` interface.
The following UML diagram shows this relationship:

R3: The system should allow guests to search for and book any available room.

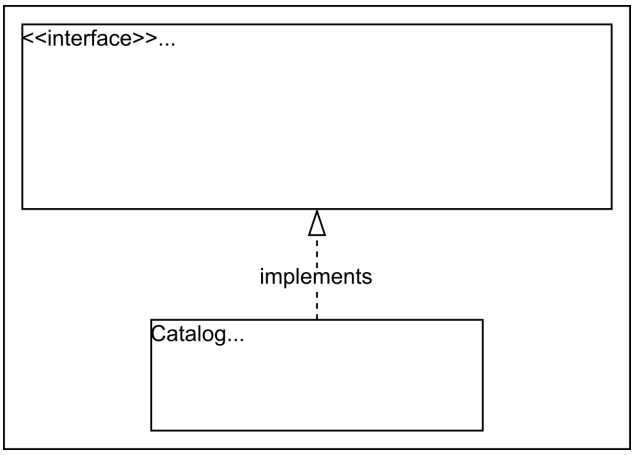


Figure 1.8: The class diagram of the Search interface and the Catalog class

Bill transaction

After generating an invoice, a customer must pay the bill to confirm the room booking. A `BillTransaction` class is required to store the information of bill payment. Check , cash, and credit card transactions are three ways to pay the bill. We can define the bill payment functionality through any payment method using the diagram below:

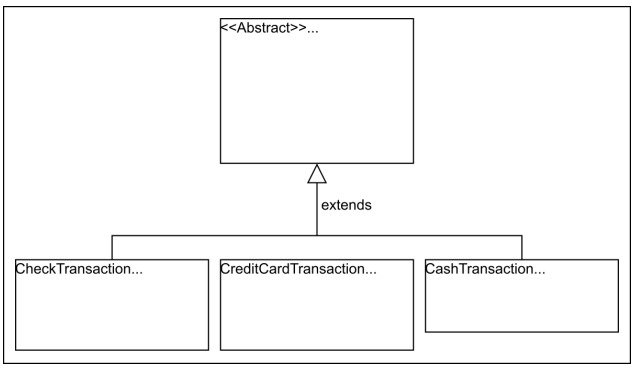


Figure 1.9: The class diagram of BillTransaction and its derived classes

Hotel and hotel branch

In this section, we'll look at the `Hotel` and `HotelBranch` classes. According to the requirements, there can be multiple branches of the hotel. `HotelBranch` is a class used to represent the location of the hotel branch. This class consists of two members: `name` and `Address`. The string type `name` is used to store the name of the hotel branch, while the complex object `Address` is used to store the complete address of a branch.
The `Hotel` class is the base class of the system, which is used to represent the hotel. The visual representation of these classes is as follows:

R10: A hotel can have multiple branches.

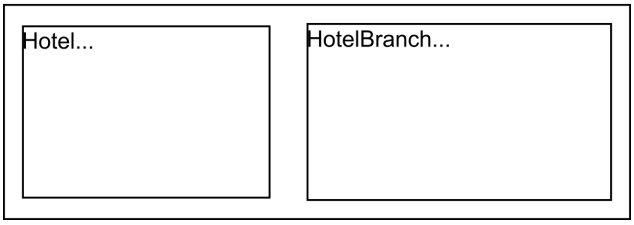


Figure 1.10: The class diagram of the Hotel and HotelBranch classes

Enumerations

Here is the list of enumerations required in the hotel management system:
`BookingStatus:` This status describes the booking status, whether the booking is requested, pending, confirmed, canceled, or abandoned.
`PaymentStatus:` This status describes the status of the booking's payment, whether it is unpaid, pending, completed, failed, declined, canceled, abandoned, setting, settled, or refunded.
`RoomStatus:` This status describes the room's status, whether it is available, reserved, occupied, not available, being serviced, or any other possibility.
`RoomStyle:` This describes the style of the room that the user wants to book. The style could be standard, deluxe, family, or business.
`AccountStatus:` This status tells the status of the user account, whether it is active, closed, canceled, or blocklisted.
`AccountType:` The account type tells the type of the user's account, whether it is a member, guest, manager, or receptionist.

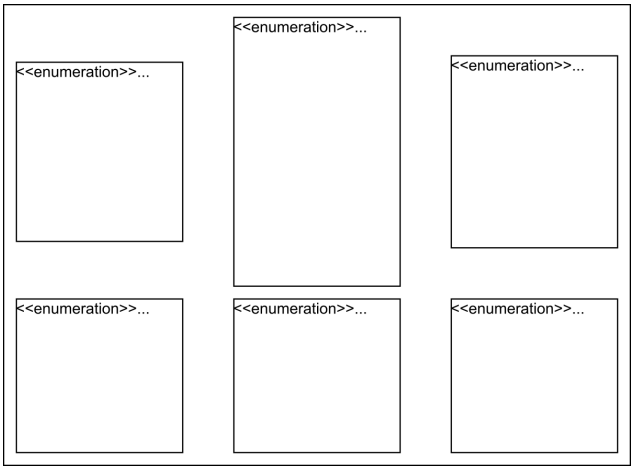


Figure 1.11: Enums in the hotel management system

R2: The rooms can be of different styles, such as standard, deluxe, family suite, or business suite.

4.2 Relationship between the classes

Now, we'll discuss the relationships between the classes we have defined above in our hotel management system.

Association

The class diagram has the following association relationships:

One-way association

- The `Housekeeper` class has a one-way association with `RoomMaintenance`.
- Both `Receptionist` and `Guest` have a one-way association with `RoomBooking`.
- The `RoomBooking` class has a one-way association with `Room`.

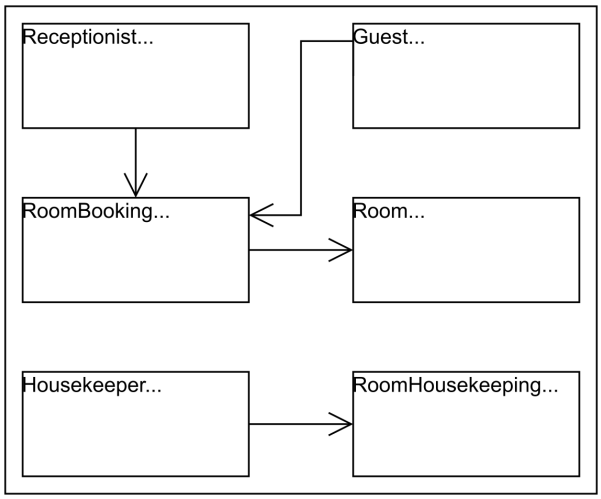


Figure 1.12: A one-way association relationship between classes

Two-way association The `RoomBooking` class has a two-way association with `Notification`.

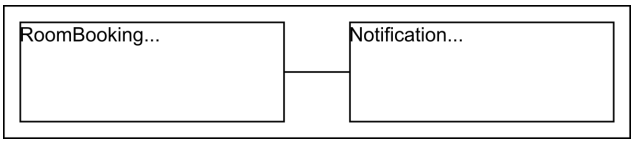


Figure 1.13: A two-way association relationship between classes

Aggregation

- The `RoomBooking` class is an aggregate of `Service`.

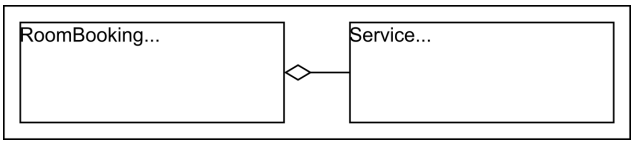


Figure 1.14: An aggregation relationship between classes

Composition

- The `RoomBooking` class is composed of `Invoice`.
- The `Invoice` class is composed of `BillTransaction`.
- The `Person` class is composed of `Account`.
- The `Room` class is composed of `RoomMaintenance` and `RoomKey`.
- The `HotelBranch` class is composed of `Room`.
- The `Hotel` class is composed of `HotelBranch`.

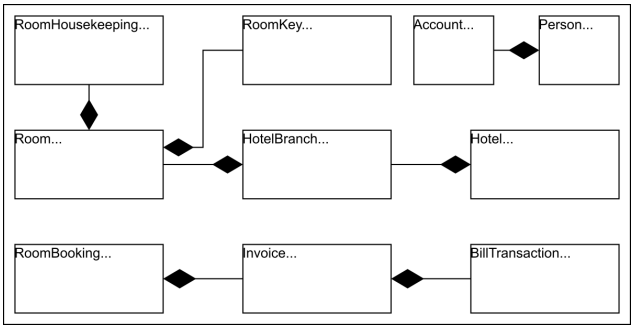


Figure 1.15: A composition relationship between classes

Inheritance

The following classes show an inheritance relationship:

- `Receptionist`, `Guest`, and `Housekeeper` classes extend the `Person` class.
- `Amenity`, `RoomService`, and `KitchenService` classes extend the `Service` class.
- Both `PostalNotification` and `EmailNotification` classes extend the `Notification` class.
- The `Catalog` class implements the `Search` interface.

  Note: We have already discussed the inheritance relationship between classes in the component section above one by one.

4.3 Class diagram of the hotel management system

This section outlines the multiplicity (cardinality) relationships between the main classes in our hotel management system. We explain the allowed number of instances on each side for each relationship and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

Source	**Target**	**Multiplicity**	**Reason**
Hotel	HotelBranch	1–0..*	A single hotel can have multiple branches across different locations.
HotelBranch	Room	1–0..*	Each branch contains multiple rooms for booking.
Room	RoomBooking	1–0..*	A room can be booked multiple times over its lifetime.
Guest	RoomBooking	1–0..*	A guest may have multiple bookings over time.
Room	RoomMaintenance	1–0..*	A room may undergo multiple maintenance sessions.
HouseKeeper	RoomMaintenance	1–0..*	A housekeeper may be assigned to multiple room maintenance tasks.
RoomBooking	Notification	1–0..*	Each booking can trigger multiple notifications (email or SMS).
RoomBooking	Invoice	1–1	Each booking results in exactly one invoice being generated.
Invoice	BillTransaction	1–1..*	One invoice may involve one or more payment transactions.
Catalog	Room	1–0..*	A catalog maintains a list of multiple rooms available for booking.
Room	RoomKey	1–0..*	A room may have multiple keys (digital or physical).
Account	Person (Guest, etc.)	1–1	Each account is tied to exactly one person entity and vice versa.

Table 1.1: Entity Relationships and Multiplicities in a Hotel Management System

Here is the complete class diagram for our hotel management system:

4.4 Design pattern

We apply the object-oriented design patterns based on system behavior to implement hotel management systems' core features in a flexible and scalable way.

- The hotel system allows guests to receive notifications (like booking confirmations or reminders). The Observer design pattern can be used to model this behavior.

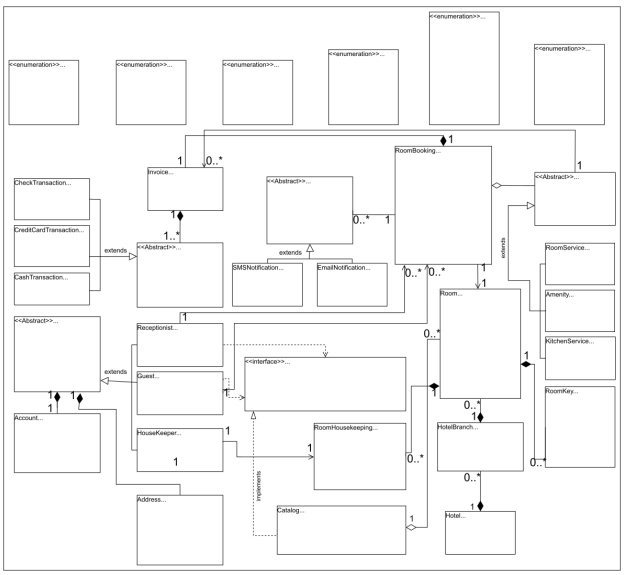


Figure 1.16: The class diagram of the hotel management system

- The hotel system supports payment methods such as credit card, cash, and check. To handle this flexibly, we can use the Strategy design pattern.

- Booking confirmations may need to trigger different notifications (SMS or Email). We can use the Factory design pattern to manage these interchangeable notification types.

- We know that each payment type (cash, credit, check) follows a similar flow, but with some specific steps. We can use the Template Method design pattern to structure this reusable logic.

- We know that the catalog needs to manage and search through a collection of individual room objects. We can use the Composite design pattern to treat individual and grouped room operations uniformly.

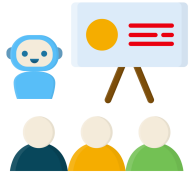
- We know that the catalog or hotel instance should be globally accessible and should not have multiple copies. To enforce a single instance, we can use the Singleton design pattern.

- Room bookings and invoices involve setting multiple details, such as dates, rooms, guests, and status. The Builder design pattern can help us manage this complex construction clearly and safely.

- We know that components like notification services or payment processors might change or need mocking in tests. To make the system more flexible and testable, we can use Dependency Injection.

4.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.



4.6 Additional requirements

The interviewer can introduce some additional requirements in the given hotel management system, or they can ask some follow-up questions. Let 's see examples of the additional requirements:

Discount: Depending on special events such as the New Year, branch opening, a discount will be applied to the payment. The class diagram provided below shows the relationship of `Discount` with the `BillTransaction` class:

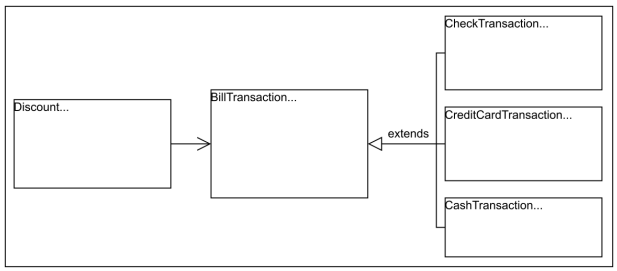


Figure 1.17: The relationship of the Discount class with the BillTransaction class

We have successfully designed the class diagram for the hotel management system. Let 's move to the next lesson to see how we can construct the sequence diagram for the system.

5 Sequence Diagram for the Hotel Management System

Sequence diagrams are a great way to understand the interactions between different entities and objects in the system. There can be different sequence diagrams that we can create for our hotel management system. In this lesson, we will create sequence diagrams for the following two interactions:

- **Book a room:** The guest books a hotel room online.

- **Sequence challenge:** The guest checks out of their room at the reception.

5.1 Book a room

The sequence diagram for the room booking should have the following actors and objects that will interact with each other:

- **Actor:** `Guest`
- **Objects:** `Catalog`, `Booking`, `Room`, and `Payment`
- System

Here are the steps in the book room interaction:

1. The guest searches for a room based on price and style.
2. The catalog returns a list of rooms.
3. The guest selects a room they wish to book.
4. If the room is available:
   (a) The guest creates a booking for the room.
   (b) The booking fetches the booking price for the room.
   (c) The guest is informed that the booking is ready for payment.
   (d) The guest initiates a payment against the booking price.
   (e) The payment is processed, and the guest is informed of the status.
   (f) If the payment is successful:
      i. The guest is informed that the payment has succeeded.
      ii. The system is informed that payment is complete.
      iii. The system updates the room status to reserved.
   (g) Else if the payment is unsuccessful:
      i. The guest is informed that the payment has failed.
5. Else if the room is unavailable:
   (a) The system informs the guest that the room is unavailable.

Based on the order above, the sequence diagram of booking a room in a hotel management system is given below:

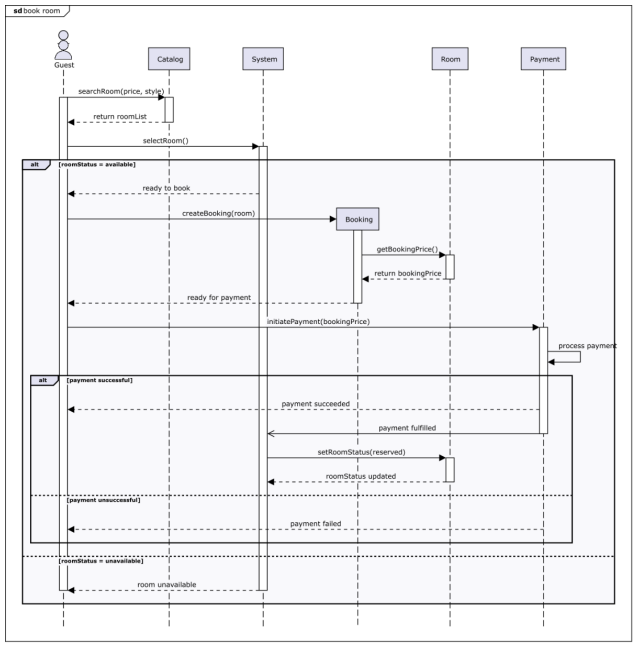


Figure 1.18: The sequence diagram for online room booking

5.2 Sequence challenge: The housekeeper performs maintenance in a room

You will help us complete a sequence diagram for the housekeeper who performs maintenance in a room.
A skeleton of the sequence diagram is provided below.

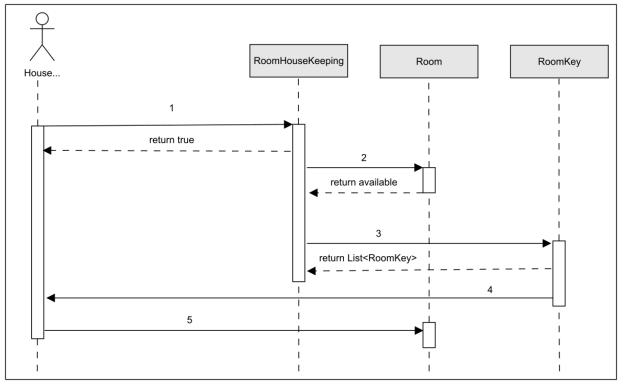


Figure 1.19: The sequence diagram for checking out of the hotel

Notice that the arrows in the diagram above are numbered from 1 to 5. Below are the messages between the actor(s) and object(s). Can you rearrange the messages below in the correct order sequence they should appear in the skeleton of the sequence diagram above?

Note: If you're stuck, click the "Show Solution" button.

In the next lesson, we'll create activity diagrams for the hotel management system.

6 Activity Diagram for the Hotel Management System

Activity diagrams are a great way to visualize the flow of messages from one activity to another in the system. There can be different activity diagrams that we can create for our hotel management system. In this lesson, we'll create activity diagrams for the following two activities:

- Hotel check-in.
- **Activity challenge:** Cancel a room booking.

6.1 Hotel check-in

The following are the states and actions that will be involved in this activity diagram.

States

Initial state: A guest with a room booking comes to the hotel reception for check-in.
Final state: The guest successfully checked in at the hotel.

Actions

The guest has a room booked and arrives at the hotel reception. The receptionist validates the booking and checks if the room is ready. The receptionist then issues room keys and updates the room status.
Based on the order above, the activity diagram of the hotel check-in is shown below.

6.2 Activity challenge: Cancel a room booking

You'll help us create an activity diagram of a customer who wants to cancel their room booking.
A skeleton of the activity diagram is given below, provided that a guest will cancel the room booking from the hotel.

Notice that the actions in the diagram above are numbered from 1 to 4. The slots below represent the activities, and the arrows represent the flow from one activity to another. Can you rearrange the slots below in the correct order they should appear in the activity diagram above?

> **Note:** If you're stuck, click the "Show Solution" button.

Alternatively, you can click the "Show complete diagram" button below to see the complete activity diagram.

We've looked at some of the activity diagrams of our hotel management system. In the next lesson, we will present the code for our designed classes in some of the most popular languages.

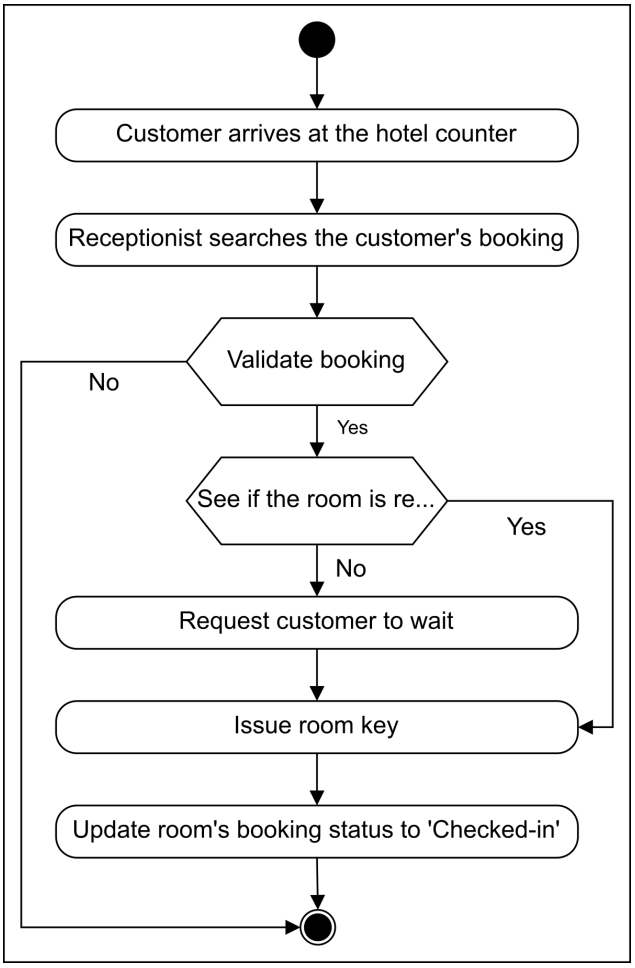


Figure 1.20: The activity diagram for the hotel check-in

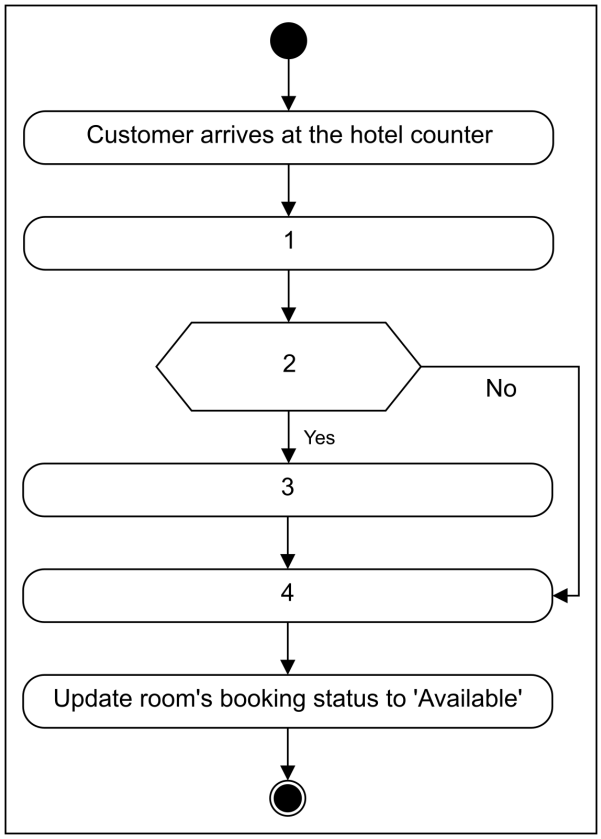


Figure 1.21: The activity diagram for booking cancellation

7 Code for the Hotel Management System

We've reviewed different aspects of the hotel management system and observed the attributes attached to the problem using various UML diagrams. Let 's explore the more practical side of things, where we will work on implementing the hotel management system using multiple languages. This is usually the last step in an object-oriented design interview process.
We have chosen the following languages to write the skeleton code of the different classes present in the hotel management system:

- Java
- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click Executable Code: Hotel Management System.

7.1 Hotel management system classes

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public methods.

Enumerations

First, we will define all the enumerations required in the hotel management system. According to the class diagram, six enumerations are used in the system, i.e., `RoomStyle`, `RoomStatus`, `BookingStatus`, `AccountStatus`, `AccountType`, and `PaymentStatus`. The code to implement these enumerations is as follows:

> **Note:** JavaScript does not support enumerations, so we will use the Object.freeze() method as an alternative that freezes an object and prevents further modifications.

Enum definitions

```
// definition of enumerations used in hotel management system
enum RoomStyle {
    STANDARD,
    DELUXE,
    FAMILY_SUITE,
```

```
    BUSINESS_SUITE
}

enum RoomStatus {
    AVAILABLE,
    RESERVED,
    OCCUPIED,
    NOT_AVAILABLE,
    BEING_SERVICED,
    OTHER
}

enum BookingStatus {
    REQUESTED,
    PENDING,
    CONFIRMED,
    CANCELLED,
    ABANDONED
}

enum AccountStatus {
    ACTIVE,
    CLOSED,
    CANCELED,
    BLACKLISTED,
    BLOCKED
}

enum AccountType {
    MEMBER,
    GUEST,
    MANAGER,
    RECEPTIONIST
}

enum PaymentStatus {
    UNPAID,
    PENDING,
    COMPLETED,
    FILLED,
    DECLINED,
    CANCELLED,
    ABANDONED,
    SETTLING,
    SETTLED,
    REFUNDED
}
```

Address and account

This section contains the `Address` and `Account` classes. Here , the `Address` class is used as a custom data type. The implementation of these classes is shown below:

The Address and Account classes

```
public class Address {
    private String streetAddress;
    private String city;
    private String state;
    private int zipCode;
    private String country;
}

public class Account {
    private String id;
    private String password;
    private AccountStatus status;

    public boolean resetPassword();
}
```

Person

`Person` is an abstract class that represents the various people or actors that can interact with the system. There are four types of persons: `Guest`, `Receptionist`, `Server`, and `Housekeeper`. The implementation of the mentioned classes is shown below:

Person and its child classes

```
public abstract class Person {
    private String name;
    private Address address;
    private String email;
    private String phone;
    private Account account;
}


public class Guest extends Person {
    private int totalRoomsCheckedIn;

    public List<RoomBooking> getBookings();
    public boolean createBooking();
}

public class Receptionist extends Person {
    public List<Person> searchMember(String name);
    public boolean createBooking();
}

public class Housekeeper extends Person {
    public boolean assignToRoom();
}
```

Service

`Service` is an abstract class, and this section represents different charges (`Amenity`, `RoomService`, and `KitchenService`) against a booking. The code to implement these classes is shown below:

Service and its derived classes

```
public abstract class Service {
    private Date issueAt;

    public boolean addInvoiceItem(Invoice invoice);
}

public class Amenity extends Service {
    private String name;
    private String description;
}

public class RoomService extends Service {
    private boolean isChargeable;
    private Date requestTime;
}

public class KitchenService extends Service {
    private String description;
}
```

Invoice

This section contains information on the `Invoice` class to create a bill. The implementation of this class is represented below:

The Invoice class

```
public class Invoice {
    private double amount;

    public boolean createBill();
}
```

Room booking

`RoomBooking` is a class responsible for managing the bookings for a room. The implementation of this class is given below:

The RoomBooking class

```
public class RoomBooking {
    private String reservationNumber;
    private Date startDate;
    private int durationInDays;
    private BookingStatus status;
    private Date checkin;
    private Date checkout;

    private int guestId;
    private Room room;
    private Invoice invoice;
    private List<Notification> notifications;

    public static RoomBooking fectchDetails(String reservationNumber);
}
```

Bill transaction

After generating an invoice, a customer must pay the bill to confirm the room booking. A `BillTransaction` class is required to store the information about bill payment. Three ways to pay the bill are check, cash, and credit card transactions.

BillTransaction and its derived classes

```
// BillTransaction is an abstract class
public abstract class BillTransaction {
    private Date creationDate;
    private double amount;
    private PaymentStatus status;

    public abstract void initiateTransaction();
}

class CheckTransaction extends BillTransaction {
    private String bankName;
    private String checkNumber;

    public void initiateTransaction() {
        // functionality
    }
}

class CreditCardTransaction extends BillTransaction {
    private String nameOnCard;
    private int zipcode;

    public void initiateTransaction() {
        // functionality
    }
}

class CashTransaction extends BillTransaction {
```

```
    private double cashTendered;

    public void initiateTransaction() {
        // functionality
    }
}
```

Notification

The `Notification` class is another abstract class responsible for sending notifications, with the `SMSNotification` and `EmailNotification` classes as its children. The implementation of this class is given below:

Notification and its derived classes

```
// Notification is an abstract class
public abstract class Notification {
    private int notificationId;
    // The Date data type represents and deals with both date and time.
    private Date createdOn;
    private String content;

    public abstract void sendNotification(Person person);
}

class SMSNotification extends Notification {

    public void sendNotification(Person person) {
        // functionality
    }
}

class EmailNotification extends Notification {

    public void sendNotification(Person person) {
        // functionality
    }
}
```

Room, room key, and room maintenance

The `Room` class represents a room in the hotel. `RoomKey` is a class used to express the electronic key card, and `RoomMaintenance` is a class used to keep track of all the maintenance records for the rooms. The implementation of these classes is given below:

The Room, RoomKey, and RoomMaintenance classes

```
public class Room {
```

```
    private String roomNumber;
    private RoomStyle style;
    private RoomStatus status;
    private double bookingPrice;
    private boolean isSmoking;
    private List<RoomKey> keys;
    private List<RoomHousekeeping> housekeepingLog;

    public boolean isRoomAvailable();
    public boolean checkin();
    public boolean checkout();
}

public class RoomKey {
    private String keyId;
    private String barcode;
    private Date issuedAt;
    private boolean isActive;
    private boolean isMaster;

    public boolean assignRoom(Room room);
}

public class RoomHousekeeping
{
    private String description;
    private Date startDatetime;
    private int duration;
    private Housekeeper housekeeper;

    public boolean addHousekeeping(Room room);
}
```

Search and catalog

`Search` is an interface, and the `Catalog` class implements the search interface to help in the room search. The code to perform this functionality is presented below:

The Search interface and Catalog class

```
public interface Search {
    public static List<Room> search(RoomStyle style, Date date, int duration);
}

public class Catalog implements Search {
    private List<Room> rooms;

    public List<Room> search(RoomStyle style, Date date, int duration);
}
```

Hotel and hotel branch

The `Hotel` class is the base class of the system that represents the hotel. The implementation of these classes is given below:

The HotelBranch and Hotel classes

```
public class HotelBranch {
  private String name;
  private Address location;

  public List<Room> getRooms();
}

public class Hotel {
  private String name;
  private List<HotelBranch> locations;

  public boolean addLocation(HotelBranch location);
}
```

7.2 Executable code: Hotel management system

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the core workflows of the Hotel Management system. The main driver code of the system resides in the `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for all respective languages. You can click the "Run" button to execute the codes.

What does this code show

This code simulates a basic hotel management system workflow involving hotel setup, room booking, notifications, and billing. It demonstrates how different components—such as hotels, branches, rooms, guests, bookings, and transactions—interact within the system in a real-world scenario.

- **Scenario 1 (Hotel and room setup):** A hotel named "Ocean View Resort" has two branches: the *Beachside* and *City Center*. A catalog is initialized with two rooms—one standard and one deluxe. This forms the foundational setup for booking and operational activities.
- **Scenario 2 (Room search, booking, and notification):** The system searches for available "STANDARD" rooms. A guest named Alice Smith is created, and a booking is made for one of the available rooms. Email and SMS notifications are triggered and sent to the guest upon successful booking, showcasing customer communication capabilities.
- **Scenario 3 (Payment and checkout process):** A cash transaction is initiated for the guest's stay, marking the billing step of the process. After the stay is complete,

the room is checked out, and its status is updated. This scenario represents a guest's interaction with the hotel—from booking to payment and checkout.

Hint: Try customizing the code!
This code is designed for experimentation, not just reading. You can edit, extend, or modify any part of the example.

Hotel Management System

```
import java.util.Date;
import java.util.List;
import java.util.ArrayList;

public class Driver {
    public static void main(String[] args) {
        // ==============================
        //  Scenario 1: Hotel and Room Setup
        // ==============================
        System.out.println("=== Scenario 1: Hotel and Room Setup ===");

        // Step 1: Create the hotel
        Hotel hotel = new Hotel("Ocean View Resort");
        System.out.println("Hotel created: " + hotel.getName());

        // Step 2: Add branches to the hotel
        HotelBranch beachBranch = new HotelBranch("Beachside Branch", new Address())
            ;
        HotelBranch cityBranch = new HotelBranch("City Center Branch", new Address()
            );

        hotel.addLocation(beachBranch);
        hotel.addLocation(cityBranch);
        System.out.println("Added " + hotel.getLocations().size() + " hotel branches
            .");

        // Step 3: Create and add rooms
        Room room101 = new Room("101", RoomStyle.STANDARD, RoomStatus.AVAILABLE,
            120.0, false);
        Room room202 = new Room("202", RoomStyle.DELUXE, RoomStatus.AVAILABLE,
            220.0, true);

        Catalog catalog = new Catalog();
        catalog.getRooms().add(room101);
        catalog.getRooms().add(room202);
        System.out.println("Room catalog initialized with " + catalog.getRooms().
            size() + " rooms.
");

        // ==============================
        //  Scenario 2: Room Search, Booking & Notification
        // ==============================
        System.out.println("=== Scenario 2: Room Search, Booking & Notification ==="
            );
```

```
// Step 1: Search for STANDARD rooms
System.out.println("Searching for available STANDARD rooms for 2 nights...")
    ;
List<Room> availableRooms = catalog.search(RoomStyle.STANDARD, new Date(),
    2);
System.out.println("Available STANDARD rooms found: " + availableRooms.size
    ());

// Step 2: Guest creation and room booking
Guest guest = new Guest();
guest.setName("Alice Smith");
guest.setEmail("alice.smith@example.com");
guest.setPhone("987-654-3210");

RoomBooking booking = new RoomBooking();
booking.setReservationNumber("RS789123");
booking.setRoom(room101);
booking.setStartDate(new Date());
booking.setDurationInDays(2);
booking.setStatus(BookingStatus.CONFIRMED);

guest.getBookings().add(booking);
System.out.println("Booking confirmed for guest: " + guest.getName());
System.out.println("Reservation Number: " + booking.getReservationNumber());

// Step 3: Notifications
EmailNotification email = new EmailNotification();
email.setNotificationId(101);
email.setContent("Hello " + guest.getName() + ", your room booking is
    confirmed!");
email.setCreatedOn(new Date());
email.sendNotification(guest);

SMSNotification sms = new SMSNotification();
sms.setNotificationId(102);
sms.setContent("Hi " + guest.getName() + ", booking confirmed. Enjoy your
    stay!");
sms.setCreatedOn(new Date());
sms.sendNotification(guest);

System.out.println();

// ================================
//  Scenario 3: Payment & Checkout Process
// ================================
System.out.println("=== Scenario 3: Payment & Checkout Process ===");

// Step 1: Initiate payment
BillTransaction transaction = new CashTransaction();
transaction.setAmount(room101.getBookingPrice());
transaction.setCreationDate(new Date());
transaction.setStatus(PaymentStatus.PENDING);
transaction.initiateTransaction();

System.out.println("Payment of $" + transaction.getAmount() + " initiated
    using cash.");
```

```
        System.out.println("Transaction status: " + transaction.getStatus());

        // Step 2: Room checkout
        room101.checkout();
        System.out.println("Room " + room101.getRoomNumber() + " has been
            successfully checked out.");
        System.out.println("Room status is now: " + room101.getStatus());

        System.out.println("
 All hotel operations completed successfully!");
    }
}
```

7.3 Wrapping up

In this chapter, we've explored the complete design of a hotel management system. We 've also shown that the system can be visualized using various UML diagrams and designed using object-oriented principles and design patterns.

8 Summary: Designing a Hotel Management System

Now that you've completed the hotel management system case study, let's take a moment to reflect on and consolidate what we've learned. We 'll revisit the key system requirements, identify the core classes along with their responsibilities and relationships, and highlight the major design principles applied. We 'll also examine how objects interact within the system and walk through the overall workflow to understand how the components come together to achieve the desired functionality.

8.1 Key requirements

This section outlines the primary functional requirements that guided the design of the hotel management system.

1. The system must support different user accounts: guest, receptionist, and housekeeper.
2. Rooms must be classifiable by different styles, such as standard, deluxe, family suite, or business suite.
3. Guests must be able to search for and book available rooms.
4. The booking process requires a check-in date, duration of stay, and an advance payment.
5. Bookings can be canceled with a full refund if done more than 24 hours before check-in.
6. The system needs to notify customers regarding their booking status.

7. All maintenance activities for each room must be logged and managed.
8. Guests should be able to add extra services like room service or amenities to their booking.
9. Each room must have a unique key, with a master key available for specific rooms.
10. The system must support multiple hotel branches.

8.2 System actors

Here are the primary and secondary actors who interact with the system.

- **Guest:** The primary user who can search for rooms, create, manage, cancel bookings, and make payments.
- **Receptionist:** An administrative user who can perform all guest actions, manage room inventory (add, update, remove), check guests in and out, and issue room keys.
- **Housekeeper:** Responsible for updating the maintenance status of rooms.
- **System:** A secondary actor responsible for automated tasks like sending booking notifications to guests.
- **Server:** A secondary actor that updates the status of rooms based on change requests.

8.3 Important classes and relationships

The following table identifies the core classes in the system, describing their main responsibilities and relationships.

8.4 Design highlights

This section covers the overall design methodology, the core principles followed, and the specific design patterns applied.

Design approach

The system was designed using a bottom-up approach. This involved identifying and designing the smallest, most granular components (like `Room` and `Account`), and then composing them into larger, more complex components until the entire system was constructed.

Design principles

The design of the Hotel Management system adheres to the five SOLID principles to ensure it is robust, maintainable, and scalable.

Class	Description	Important Relationships
Hotel	The main class representing the hotel, which can have multiple branches.	Composition of HotelBranch; follows the Singleton pattern.
HotelBranch	Represents a specific location or branch of the hotel.	Composition of Room.
Person	An abstract base class for all system users, holding common details such as name and address.	Composed of Account; extended by Guest, Receptionist, etc.
Guest	A user who can search for and book rooms.	Inherits from Person; associated with RoomBooking.
Receptionist	An administrative user who manages rooms and bookings.	Inherits from Person; associated with RoomBooking.
Room	Represents a single room in the hotel, with properties such as style, status, and price.	Composed of RoomKey and RoomHousekeeping.
RoomBooking	Manages reservation details, including dates, guest information, and status.	Composed of Invoice; aggregates Service.
Invoice	Represents the bill for a booking, detailing the total amount to be paid.	Composed of BillTransaction.
Service	An abstract class for additional services (e.g., room service, amenities) that can be added to a booking.	Extended by RoomService, KitchenService, and Amenity.
Notification	An abstract class for sending notifications to users.	Extended by SMSNotification and EmailNotification.
Search	An interface that defines the contract for searching rooms.	Implemented by Catalog.
Catalog	Implements the Search interface and maintains a list of all rooms available in the hotel.	Implements Search.

Table 1.2: Hotel Management System Classes and Their Relationships

- **Single Responsibility principle (SRP):** Each class has a distinct responsibility. For example, the `Room` class manages room-specific data and state, the `Invoice` class is solely responsible for billing, and the `Notification` class handles sending alerts.

- **Open/Closed principle (OCP):** The system is open for extension but closed for modification. Using abstract classes like `Person`, `Service`, and `Notification`, new types of users, services, or notification methods can be added by creating new subclasses without altering existing, tested code.

- **Liskov Substitution principle (LSP):** Subtypes can be substituted for their base types without affecting the program's correctness. For instance, a `Guest` object can be

used wherever a `Person` object is expected, and a `RoomService` can be treated as a generic `Service`.

- **Interface Segregation principle (ISP):** The `Search` interface is a clear example, providing a specific contract for room searching functionality that the `Catalog` class implements. This prevents client classes from depending on methods they don't use.
- **Dependency Inversion principle (DIP):** The system depends on abstractions rather than concrete implementations. High-level modules like `RoomBooking` interact with abstract classes like `Service`, decoupling them from the specific details of `RoomService` or `Amenity`.

Design patterns

The following design patterns were used to solve common problems and improve the system's structure.

- **Singleton:** This was used to ensure the `Hotel` or `Catalog` instance is globally accessible and that no duplicate instances are created.
- **Strategy:** This was chosen to handle different payment methods (e.g., credit card, cash) flexibly, allowing the payment algorithm to be selected at runtime.
- **Factory:** This was applied to manage creating different notification types (e.g., SMS, Email), allowing the system to create the correct type without exposing instantiation logic.
- **Observer:** This allowed guests to receive various notifications, decoupling the notification senders (like booking confirmations) from the notification receivers.
- **Builder:** This was chosen to manage the complex construction of `RoomBooking` and `Invoice` objects, which require setting multiple details like dates, guest information, and status.

8.5 Object interactions

Let's examine how objects collaborate to book a room to understand the system's dynamic behavior.

- **Room search:** A `Guest` initiates a search for a room via the `Catalog` object, which holds all room information. The `Catalog` returns a list of available rooms that match the criteria.
- **Booking creation:** Upon selecting a `Room`, the `Guest` creates a `RoomBooking`. The `RoomBooking` object then fetches the price from the selected `Room` object to calculate the total cost.
- **Payment processing:** The `Guest` initiates a `Payment` for the booking. The system processes this payment.

- **Status update:** If the payment is successful, the `System` is notified and updates the `Room`'s status to "reserved" to prevent double-booking. If the payment fails, the guest is notified, and the booking is not confirmed.

8.6 System workflow

The following describes the primary workflow for a guest checking into the hotel.

- **Arrival and verification:** A guest with a confirmed booking arrives at the reception. The `Receptionist` searches for and finds the guest's `RoomBooking` in the system.
- **Booking validation:** The receptionist validates the booking details to ensure they are correct.
- **Room readiness check:** The system checks if the assigned `Room` is clean and ready for occupancy. If not, the guest may be asked to wait.
- **Key issuance:** The `Receptionist` issues a `RoomKey` to the guest once the room is ready.
- **System update:** Finally, the receptionist updates the `Room`'s status in the system to "Checked-in," completing the check-in process.

9 Key Insights: The Hotel Management System

Well done on completing the hotel management system case study! This lesson takes you further by highlighting frequent design mistakes, sharing actionable ways to refine your design thinking, and equipping you with insights to handle tricky interview questions. You 'll also discover additional case studies to deepen your grasp of practical system design concepts.

9.1 Common mistakes

Designing the Hotel Management System can be tricky, and candidates often fall into similar traps that weaken their solutions.

- **Overlooking concurrency in bookings:** Many candidates forget to handle double-booking scenarios. For example, two guests could book the same room if overlapping bookings aren't properly synchronized. Design the `RoomBooking` logic so that checking a room's availability and marking it as booked happen together as one single step. This ensures that no other booking can claim the same room simultaneously.
- **Unclear separation of responsibilities:** It's common to mix roles, such as letting the `Guest` class handle room status updates instead of the `Receptionist` or `System` classes. Keep each class focused on its responsibility. For example, the `Guest` only requests actions; `receptionists` and `Systems` update room status and issue keys.

- **Ignoring edge cases for cancellations:** Candidates often neglect refund rules, such as the requirement that full refunds be applied only if a booking is canceled at least 24 hours before check-in. Clearly define cancellation rules in the `RoomBooking` logic and update `Invoice` and `PaymentStatus` accordingly to handle partial or no refunds.

- **Missing proper pricing strategy handling:** Some candidates hard-code room prices and forget that different rooms may require flexible pricing models (e.g., seasonal rates or special discounts). Use the `Strategy` pattern to separate and switch pricing algorithms easily, so each room type can apply its pricing logic when needed.

- **Overlooking hotel hierarchy relationships:** Many designs treat the hotel as a single unit and don't model how multiple branches, each with its rooms and bookings, fit into the bigger structure. Use the `Hotel` and `HotelBranch` classes to represent a clear hierarchy supporting multiple branches, each managing its inventory independently.

9.2 Key interview tips

Small changes in how you present your design can make a big difference. Here are tips to communicate your Hotel Management System design effectively:

Tip	**Explanation**
Prioritize user flows.	Before creating class diagrams, outline the most critical user flows (e.g., booking a room, checking in). This demonstrates a strong understanding of requirements and sets the context for the design, making it easier for interviewers to follow the thought process.
Explain account types.	Clarify how Guest, Receptionist, and Housekeeper extend the base Person class to keep responsibilities clear and modular.
Discuss notifications.	Explain how notifications (SMS and Email) are handled through the Notification abstract class to keep guests informed automatically.
Highlight system extensibility.	Describe how adding new room styles, services, or payment types does not break existing classes due to the use of abstract classes and clear interfaces.
Mention fail-safe flows.	Discuss how the design handles failures, such as payment failure during booking or abandoned booking scenarios.

Table 1.3: Design and Explanation Tips for a Hotel Management System

9.3 Test your understanding

Test your knowledge with these questions and confirm you understand how to design and explain this system well.

The Hotel Management System

Question 1: Which relationship best describes how `RoomBooking` relates to `Invoice`?

- Association
- Aggregation
- **[CORRECT]** Composition
  Explanation: The `RoomBooking` class is composed of the `Invoice` class, i.e., it cannot exist independently
- Generalization

Question 2: Which principle does extending the `Person` class into subclasses `Guest`, `Receptionist`, and `Housekeeper` best represent?

- Open/Closed principle
- **[CORRECT]** Single Responsibility principle
  Explanation: Each subclass has clear, separate duties
- Liskov Substitution principle
- Dependency Inversion principle

Question 3: Which design element helps manage different room services like amenities and kitchen orders?

- The `Room` class directly handles them
- **[CORRECT]** A `Service` abstract class with child classes
  Explanation: A `Service` abstract class with subclasses `Amenity`, `RoomService`, and `KitchenService` manages extra services.
- The `Catalog` class
- The `Account` class

Question 4: Why does the design use a Strategy pattern for room pricing instead of putting price logic inside the `Room` class?

- To increase inheritance hierarchy depth.
- To keep the `Room` class open for modification.
- **[CORRECT]** To follow the Open/Closed principle and allow new pricing algorithms without changing `Room`.

  Explanation: Using Strategy keeps the `Room` class closed for modification but open for extension with new pricing logic.
- To reduce the number of classes.

Question 5: If the `Receptionist` checks in a guest but forgets to issue a `RoomKey`, which part of the design could detect or prevent this inconsistency?

- **[CORRECT]** The `Housekeeper` class should check the key status.
- The `Hotel` Singleton should auto-issue keys.
- The `Search` interface should reject the booking.
- The `RoomBooking` process should include an explicit check-key step.

  Explanation: The `RoomBooking` flow should enforce a step to check that a key is issued when check-in is done. Otherwise , the room can't be securely accessed.

9.4 Related case studies

Explore these related case studies to deepen your understanding of similar design challenges and reusable design ideas.

- **Designing a car rental system:** Similar to hotel room booking, this case study tests your approach to handling availability, overlapping bookings, dynamic pricing, and branch-specific inventory.
- **Designing an airline management system:** Reinforces concepts like seat reservation conflicts, cancellation policies, and notification flows, just like hotel room booking.
- **Designing a restaurant management system:** This will help you practice handling table reservations, service requests, staff roles, and payments, closely mirroring hotel dining or room service add-ons.
- **Designing a movie ticket booking system:** Strengthens your thinking about time-slot-based booking, seat selection, preventing double bookings, and handling online payments.

- **Designing the Amazon online shopping system:** Builds on the idea of handling multiple user accounts, dynamic pricing strategies (discounts, special offers), and payment workflows similar to hotel service add-ons and billing.